

Архитектура этой системы должна одновременно справляться с экстремальным Highload на запись (миллионы водителей ежесекундно шлют GPS) и гарантировать строгую финансовую консистентность (нельзя списать деньги дважды или выдать один заказ двум водителям).

Этап 1: Вопросы для уточнения (Сбор требований)

1. Функциональные:

- *Вопрос:* Какие основные сценарии мы покрываем?
- *Ответ:* 1. Пассажир видит машины на карте и заказывает поездку.
2. Водитель постоянно передает гео-координаты, получает и выполняет заказы.
- *Вопрос:* Как работает подбор водителя (матчинг)?
- *Ответ:* На основе минимального времени подачи (ETA) с учетом дорожного графа и пробок.
- *Вопрос:* Нужен ли расчет стоимости на лету?
- *Ответ:* Да, цена рассчитывается динамически при заказе (с учетом Surge Pricing — повышающих коэффициентов при высоком спросе).

2. Нефункциональные:

- *Вопрос:* Какой масштаб системы (DAU / Активные водители)?
 - *Ответ:* 15 млн активных пассажиров в день (DAU). 1.5 млн активных водителей на линии одновременно в пике.
 - *Вопрос:* Какие требования к задержкам (Latency)?
 - *Ответ:* Обновление машин на карте пассажира — до 2 секунд. Назначение машины — до 5 секунд.
 - *Вопрос:* Что важнее: доступность или консистентность (CAP)?
 - *Ответ:* Система гибридная. Для гео-трекинга важна высокая доступность (**AP**), потеря пары координат не критична. Для биллинга и статусов заказов важна строгая консистентность (**CP**).
-

Этап 2: Расчеты нагрузок (Estimations)

- **Трафик на запись гео-данных (Write QPS):**

1.5 млн водителей отправляют координаты (`latitude`, `longitude`, `driver_id`, `timestamp`) каждые 4 секунды.

$$\backslash(QPS=1\ 500\ 000/4=\mathbf{375\ 000})\ \text{запросов в секунду}\backslash$$

- **Трафик на создание заказов (Order QPS):**
Пусть 15 млн пассажиров делают по 2 поездки в день = 30 млн поездок. Распределим пики (утро/вечер), заложив 20% поездок на 2 часа пика.
$$\text{Пиковый QPS заказов} = (30\,000\,000 \times 0.2) / 7200 \approx \mathbf{833 \text{ заказов в секунду}}$$
 - **Объем хранилища для гео-логов траекторий (Storage):**
Размер одного пакета координат (≈ 64) байта. В секунду: $(375\,000 \times 64 \text{ байт}) \approx 24 \text{ МБ/сек}$.
За 15-часовую рабочую смену всех водителей: $(24 \text{ МБ} \times 3600 \times 15) \approx \mathbf{1.3 \text{ ТБ чистых гео-логов в день}}$. Для этого потребуется долгосрочное OLAP-хранилище (ClickHouse).
 - **Память для текущего статуса водителей (RAM):**
Нам нужно держать в оперативной памяти *только последние координаты* активных водителей.
 $(1\,500\,000 \times 128 \text{ байт}) \approx 192 \text{ МБ}$. С учетом индексов сетки — до **3–5 ГБ**. Это легко помещается в RAM одного сервера Redis, но для отказоустойчивости мы его кластеризуем.
-

Этап 3: Варианты API (REST + WebSockets)

1. Клиентское API (Пассажир)

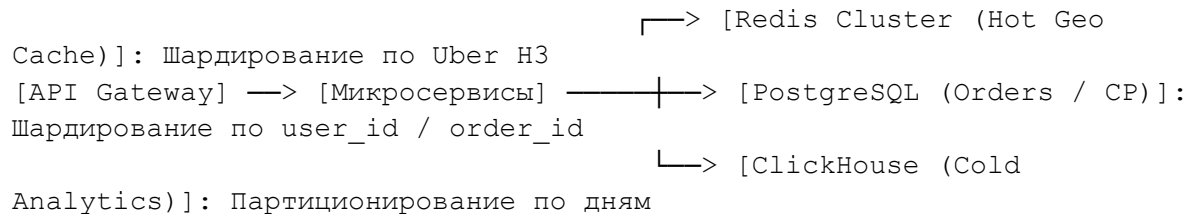
- **Создание заказа:**
`POST /api/v1/orders`
Headers: X-Idempotency-Key: <UUID>
Body: { "user_id": 999, "from": {"lat": 55.75, "lng": 37.61}, "to": {"lat": 55.79, "lng": 37.55}, "tariff": "comfort" }
Response: 201 Created -> { "order_id": "order-uuid-123", "status": "searching", "estimated_price": 450.00 }
- **Получение статуса заказа:**
`GET /api/v1/orders/order-uuid-123`
Response: { "status": "driver_assigned", "driver_info": { "name": "Иван", "car": "Kia Optima", "plate": "A777AA77" } }

2. Водительское API

- **Постоянный гео-трекинг:**
`WebSocket /ws/v1/driver/location?token=<jwt_token>`
Message (каждые 4 сек): { "lat": 55.753, "lng": 37.612, "status": "FREE", "timestamp": 1782900000 }
-

Этап 4: Системы хранения: Шардирование, Партиционирование, Индексы

Мы используем концепцию **Polyglot Persistence** (разные базы под разные задачи):



1. Слой Оперативного Гео-Кэша (Redis)

Хранит только текущие координаты и статусы водителей.

- **Шардирование:** Мир разбивается по географической сетке **Uber H3** (шестиугольники). Мы берем разрешение **H3 Resolution 6** (крупные районы) в качестве ключа шардирования (Partition Key) в Redis:
`geo:region:<h3_index>`.
- **Структура:** Внутри ключа используется **ZSET** или **Hash**. Водители Москвы пишутся в один шард Redis, водители Санкт-Петербурга — в другой. Полностью исключаются «горячие ключи» (Hot Keys).

2. Слой Заказов (PostgreSQL) — Источник правды

Требует строгой ACID-консистентности.

sql

```
CREATE TABLE orders (
  id UUID NOT NULL,
  user_id BIGINT NOT NULL,
  driver_id BIGINT,
  status VARCHAR(32) NOT NULL, -- 'CREATED', 'SEARCHING', 'ACCEPTED',
  'ARRIVED', 'IN_PROGRESS', 'COMPLETED'
  price NUMERIC(10, 2) NOT NULL,
  from_geo POINT NOT NULL,
  to_geo POINT NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  PRIMARY KEY (id, created_at)
) PARTITION BY RANGE (created_at);
```

Используйте код с осторожностью.

- **Шардирование:** База шардируется горизонтально по `user_id`.
- **Партиционирование:** Внутри каждого шарда таблица `orders` партиционируется по **диапазонам дат** (`created_at`) еженедельно. Операционной системе нужны только заказы за текущие дни, старые партиции уходят в холодный бэкап.
- **Индексы:**
 - `CREATE INDEX idx_orders_active ON orders (status) WHERE status NOT IN ('COMPLETED', 'CANCELLED');` (Частичный индекс для мгновенной выборки живых заказов).
 - `CREATE INDEX idx_orders_driver ON orders (driver_id) WHERE driver_id IS NOT NULL;`

3. Слой Аналитики и Истории Траекторий (ClickHouse)

Сюда сбрасываются все логи координат из очередей Kafka для обучения моделей машинного обучения (расчет цен и ETA).

- Таблица `driver_trajectories` использует движок `MergeTree()`.
- **Партиционирование:** `PARTITION BY toYYYYMM(timestamp)` (помесячно).
- **Сортировка (Индекс):** `ORDER BY (city_id, driver_id, timestamp)`. Позволяет за миллисекунды восстановить трек поездки водителя из миллиардов строк.

Этап 5: Подробная логика работы каждого сервиса

1. Geo Ingestion Service (Сервис приема координат)

- Принимает миллионы WebSocket-соединений от водителей.
- Выполняет быструю валидацию токена.
- Переводит координаты (`lat, lng`) в индекс ячейки **Uber H3**.
- Обновляет позицию водителя в оперативной памяти **Redis Cluster** (с TTL 30 секунд).
- Асинхронно отправляет событие `DriverLocationUpdated` в **Apache Kafka** для последующей архивации в ClickHouse.

2. Price & Surge Service (Сервис динамического ценообразования)

- Рассчитывает базовую стоимость поездки по дорожному графу между точками А и Б.

- Читает из Redis агрегированные данные по конкретной ячейке H3 пассажира: количество запросов поездок (Спрос) и количество свободных водителей (Предложение).
- Если спрос превышает предложение, применяет коэффициент (например, $\times 1.6$), стимулируя водителей ехать в этот район.

3. Demand & Matching Service (Сервис подбора водителей)

- Когда создается заказ, сервис берет H3-ячейку пассажира и запрашивает из Redis список свободных водителей в этой и 6 соседних ячейках (первый радиус гексагонов).
- Отсеивает водителей по фильтрам (класс машины, рейтинг).
- Для топ-10 ближайших по прямой машин делает запрос в **Routing Engine** (сервис на C++ с графом дорог) для расчета честного времени доезда (ETA) с учетом пробок.
- Выбирает лучшего водителя, накладывает на него распределенную блокировку в Redis (**Distributed Lock** через Redlock) на 15 секунд, чтобы ему не пришел другой заказ, и отправляет ему пуш-уведомление через Notification Service.
- Если водитель отказывается или не нажимает кнопку за 15 секунд, блокировка снимается, и сервис берет следующего водителя из списка.

Этап 6: Отказоустойчивость (Fault Tolerance)

1. **Что будет, если упадет инстанс Redis с гео-данными водителей?**
 - Пассажиры на несколько секунд перестанут видеть машинки на карте.
 - *Стратегия:* Redis развернут в режиме Sentinel / Raft Cluster с автоматическим выбором нового Master. При переключении (2-3 секунды) воркеры восстанавливают соединения. Поскольку водители шлют координаты каждые 4 секунды, новый Master-Redis мгновенно заполнится свежими, актуальными данными. Никакого тяжелого восстановления из бэкапов не требуется.
2. **Что делать при Race Condition (Два пассажира заказывают одну машину)?**
 - *Решение:* На этапе матчинга перед отправкой предложения водителю, **Matching Service** делает атомарную операцию в Redis: `SET lock:driver:<id> order:<id> NX EX 15`. Если два воркера попытаются забронировать водителя, Redis отдаст **OK** только одному. Второй воркер получит отказ и переключится на подбор другой машины.

3. Обрыв сети у водителя во время поездки:

- Таксометр в приложении водителя дублирует запись координат в локальную базу данных смартфона (SQLite). При восстановлении сети приложение отправляет пачку накопленных координат, и `Geo Ingestion Service` восстанавливает трек в ClickHouse.
-

Этап 7: Мониторинг и Обсервруемость (Observability)

1. Бизнес-метрики (Главный экран в Grafana):

- **Order Conversion Rate:** Отношение созданных заказов к успешно завершённым. Внезапное падение графиков означает сбой в матчинге или биллинге.
- **Surge Pricing Heatmap:** Мониторинг распределения коэффициентов по городу для выявления аномалий (например, коэффициент $\times 5$ в чистом поле означает баг в алгоритме подсчета спроса).

2. Системные метрики бэкенда:

- **Event Loop Lag:** Задержка асинхронного цикла в Python-сервисах (FastAPI/uvloop). Должна быть < 10 мс. Рост говорит о наличии блокирующего синхронного кода в микросервисах.
- **Kafka Consumer Lag (Топик траекторий):** Показывает, успевает ли ClickHouse записывать логи координат. Если лаг растёт, ClickHouse не справляется с дисковым I/O, требуется оптимизация размера батчей.

3. Распределенный трейсинг (Jaeger):

Каждый заказ снабжается уникальным сквозным `trace_id`. Мы можем детально отследить цепочку: сколько мс запрос на заказ провел на API Gateway -> сколько считался Surge -> сколько времени занял просчет ETA в Routing Engine -> когда пуш дошел до водителя. Это позволяет точно находить узкие места в инфраструктуре.
